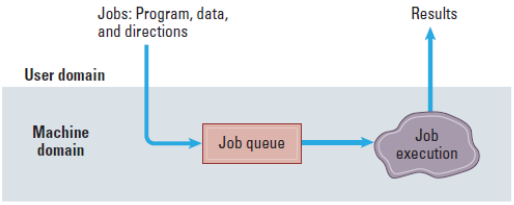
Chapter3 Operating System

Functions of Operating Systems

- **Oversee operation of computer**
  - Manages the CPU, memory ,and I/O devices.
  - Coordinates multiple **jobs**.
- **Store and retrieve files**
  - Manages data and programs in mass storage.
  - Users can use them frequently as needed.
- **Schedule programs for execution**
  - Using **job queues** and multiprogramming.
  - Determines the order and priority of jobs, supporting both batch processing and time-sharing.
- **Coordinate the execution of programs**
  - Uses multitasking, time-sharing, and multi-processor management.
  - Rapidly switches between jobs to provide interactive and real-time processing for multiple users.

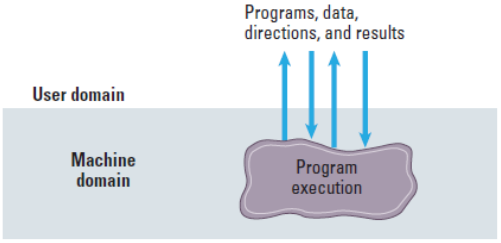
Evolution of Shared Computing

- **Batch Processing**
  - Users give jobs to the computer operator, and the operator puts all jobs into the **priority queue**. [1]
  - When the computer executes the jobs, **users cannot interact with them.**
  - **Adv**: Suitable for pre-arranged work
  - **Dis-adv:** If a user wants to change a value in the job, they cannot.



- **Interactive processing**
  - Users can communicate with the computer through **terminals** with interactions.
  - Since tasks are executed very quickly, the system requires high-performance computers.

- Adv: **Real-Time processing** → computer executes tasks in accordance with deadline.
- Dis-adv: If many users access the terminals at the same time, the overall efficiency may decrease.[2]

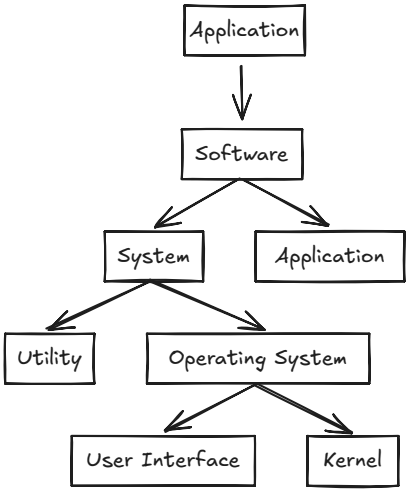


- **Time Sharing**: Multiprogramming for multiple users.
  - The CPU time is divided into multiple time intervals (time slices).
  - Each task is executed in one interval, and then switched to the next task.
- **Multitasking**: Multiprogramming for single user.

- **Multiprocessor**
  - Network Systems: Multiple computers work together through the network.
  - Embedded Systems(嵌入式系統): For dedicated equipment. `e.g. Medical device, cellphone etc`
    - Energy conservation
    - Timely processing
    - Long-term automatic operation

Operating System Architecture

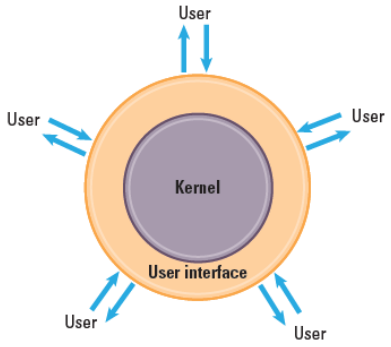
- **Type Of Software**
  - Application: Performs specific tasks for users.
  - System: Provides infrastructure for application software.
    - Utility
    - Operating System



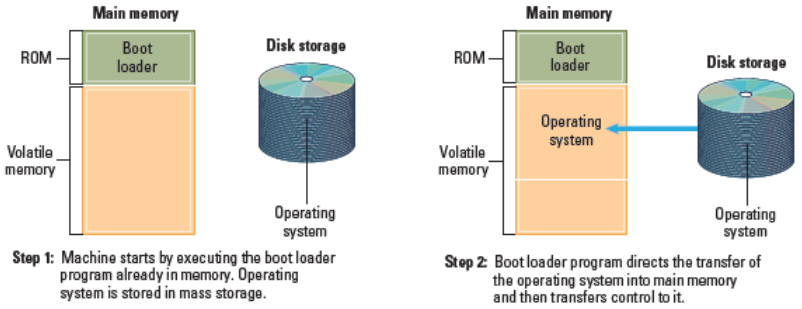
- **Components of an Operating System**
  - **User Interface**: Communicates with users.
    - Shell: **texts**
    - GUI(Graphical User Interface): Obejects are represented pictorially on the display as **icons**.
      - Window Manager: The core of the GUI.
      - **Kernel**: Performs basic computer functions.
        - File Manager: Records all files in mass storage and allows users to access those files.
        - Directory&Folder: Users can place files in a folder.
        - Directory Path: A chain of directory. `e.g. C:/Users/apps`
    - Devices Driver(驅動): Communicates with devices(like machine) through the controller.

    - Memory Manager: Coordinates the machine's use of main memory.
      - If the main memory cannot accommodate lots of tasks, the memory manager will use the concept of **virtual memory** [3]and mainly through **paging**[4] mechanism to adjust these tasks.

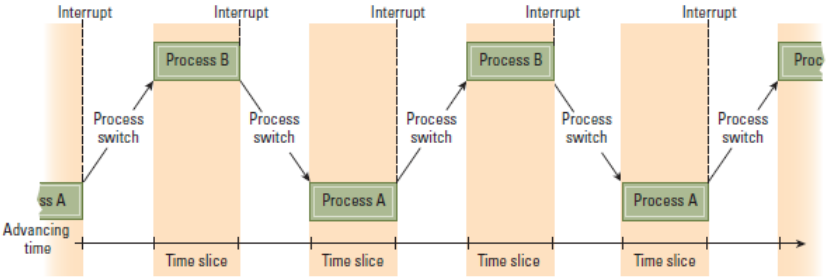
- Scheduler&Dispatcher



- **Computer Execution Process**
  - START - Bootstrapping: **CPU** executes the instruction which in the **firmware**.
    - **Boot Loader**: A program is stored and executed in **ROM**(read-only memory)[5]
      - Transfers operating system from mass storage to main memory.
      - Execute JUMP[6] to operating system.



  - EXECUTE:
    - Process: A **dynamic** activity of executing program and will change over time.
    - Program: A **static** set of directions.
    - Program State: Current status of the activity. `e.g. PC, General purpose registers, and Related portion of main memory`
- **Process Administration**
  - **Scheduler**: Maintains process table.
    - Process Table: Keeps track of all the processes.[7]
    - ADDs new process to the process table.
    - REMOVEs the completed process.
  - **Dispatcher**: Mornitors the execution of the scheduled processes.
    - **Multiprogramming:**
      - Devides time into short part which called **time slice**.
      - Each slice executes different program.
      - Changes a process to another called **Process Switch**.
      - Interruption Mechanism:
        1. When the time slice passed, CPU will **end current machine cycle** and **save current process status**.
        2. Interrupt Handler(part of dispatcher) **select the next "ready" process**.



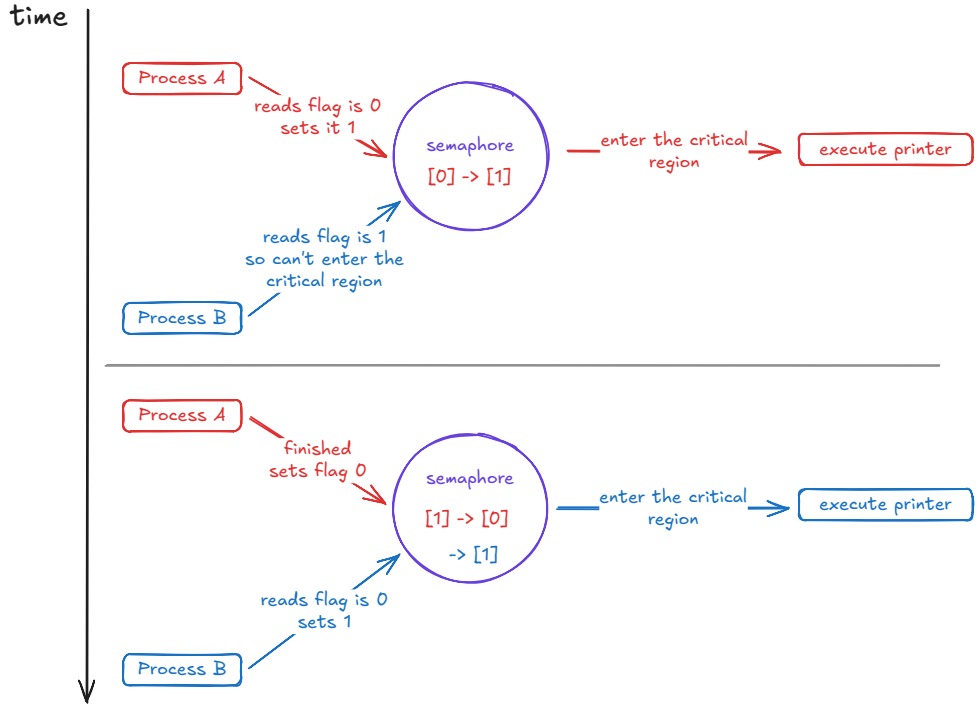
CPU Scheduling Algorithms

- **First-Come, First-Served(FCFS)**: Processes are executed in the order they arrive in the ready queue.
  - Convoy Effect: If a long-running process is executed, it will cause shorter processes to wait.
- **Shortest Job First(SJF):** Executes first the shortest process.
  - Requires knowledge of future CPU burst times, which is often unknown.

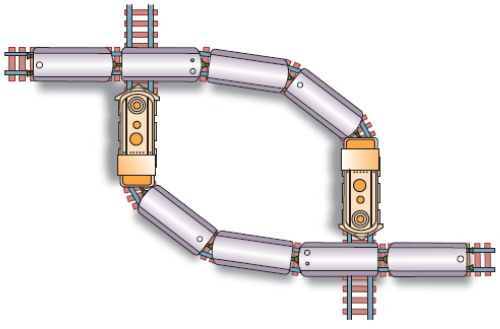
    - **Starvation**: If lots of short processes arrive continuously, long processes may not be executed.
- **Priority Scheduling**: Each process is assigned a **priority**, and the process with the highest priority is executed first.
    - **Priotiry**
        - Time Limits
        - Memory Requirements
        - I/O to CPU Burst Ratio
        - Systemic importance
    - **Starvation**: If lots of high-priority processes arrive continuously, low-priority processes may not be executed.
- **Round Robain(RR)**: Uses Time-Sharing System to assign every process to execute.
    - **Time-Sharing System**
        - Assigns every process same time slice.
        - If the process doesn't finish within its time slice, it's preempted, and the next process in the queue gets a turn.
        - **High Context Switch Overhead**: Frequent context switching can reduce performance.
- **Shortest Remaining Time First (SRTF)**: The preemptive version of SJF.
- **Multilevel Queue (MLQ):** Divides the ready queue into many queues and sorts them according to their characteristics.
- **Multilevel Feedback Queue (MLFQ):** Processes can move between different queues based on their behavior.

Handling Competition Among Processes

- **Semaphore**: A flag which **maintains** the **critical region's access control** and avoids conflicts between programs.
    - **Critical Region**: A group of instructions.
    - **Mutual Exclusion**: Requires only one process at a time be allowed to execute a critical region.



- **Dead Lock**: **Process are blocked** from executing because **each is waiting** for **a resource** currently use by another.
    - Occurrence conditions (must be simultaneous)
        1. Competition for non-sharable resources.
        2. Resources requested on a partial basis.
        3. An allocated resource can not be forcibly retrieved.
    - Resolution Methods: Solve one of three contions.
        1. Solve condition 1 : Convert nonshareable resources into shareable ones. `e.g. Spooling` [8]
        2. Solve condition 2 : Require all resources to be requested at once.
        3. Solve condition 3 : Forcibly retrieve allocated resources. `e.g. Remove(kill) one or more of the deadlocked processes.`



Security

- **Attack From Outside**
  - Auditing software: Records and analyzes the activities in the computer.
  - Sniffing software: Records activities and later reports them to a would-be intruder.
  - Unsecure password
- **Attacks from within**
  - Special-purpose register: Operating system **store each process's upper and lower of the memory area** in the register.
    - If CPU detects a process access a memory area which is **exceeds the boundary**, it will **interrupts** it.
  - Privileged mode and instructions: CPU can only **execute all instructions** under the **privileged mode**.
    - When operating system allows to **execute a process**, it will switch CPU to the **nonprivileged mode**.
    - If a process attempts to execute some privileged instructions, it will be interrupted.

---

1. If there is a job with a higher weight, it will be executed first. ↩
2. Computers in the 1960s and 1970s were expensive. ↩
3. Virtual memory provides each program with the illusion that it has a huge independent and continuous memory space (virtual address space) that is much larger than the actual available physical memory (RAM). ↩
4. Divide both virtual address space and physical memory into fixed-size blocks ↩
5. Firmware is in the ROM. ↩
6. Change the PC's address to the starting address of the operating system in RAM, so the CPU will execute the OS code from the RAM. ↩
7. Contains of address in the mamin memory, process priority, and process status. ↩
8. The operating system stores all requirements from processes in the mass storage, and when the device is available, it will transfer all data to the device. ↩